

Computer Graphics exam project descriptions

Version 1.0, 10th May 2026.

The topics of the projects are listed in a separate attachment to this document.

Projects must be done in teams of two or three students. Under special circumstances, individual projects will be allowed: the motivation for such choices should be given in the form used for registering the project. Similarly, in very special cases, even groups of four members could be allowed: this however must be discussed with the professors in advance. Every student is expected to have continuously followed the lessons and to have actively searched for at least one partner among the course mates: the motivation for going alone should explain why this search failed. All team members will have to discuss the project on the **same** exam date.

Each group will have to register for the project using a Microsoft Form whose link will be provided: in such form, you will specify your curriculum, your other team member(s), select the *topic of your project*, and add a short description of the features you would like to implement. *The link will be available in the WeBeep page of the course.*

Please keep in mind that each member will have to register individually using the form and provide the name of the other students: this helps confirming that every participant of the group is indeed interested in doing the project with the other members of the party. *Students that have not registered for the project on-line, cannot be admitted to the presentation.*

Registration for the projects will be open until mid-January 2027: there is no deadline, but it must be done before the presentation. The suggestion is to register the project when it is at about 40% of its implementation stage – i.e. when both the *group members* and *topics* are **final**. Remember that for individual projects, your registration can be denied due to lack of motivation.

Each project must be done in C++ using Vulkan, and the `Starter.hpp` used during the course. You are allowed to use AI based tools, if they give you just a minor contribution to your implementation, and if you clearly understand **EVERYTHING** that was created by the tool.

You are free to use any helper library not relevant to 3D computer graphics, for example to introduce Physics, handle collisions or include IA. Since however we must be able to compile your code, it should be something that can be easily found online and installed, it should be included with your delivery, and it must be open source. If in doubt, please ask before basing your code on some material found on-line.

Projects will have to showcase what you have learnt during the course. Your code will need to have sections where it explicitly loads or creates the geometries and the textures, it sets the shaders (*which must be written by yourself*), it sets the vertex formats and the uniform blocks, it creates the graphics pipelines and the render passes, it allocates the command buffers, and it records the draw-calls to finally show the scene on screen.

Your application will show objects that are defined through a set of local coordinates, which are mapped to the screen after a set of transformations. They must first undergo a local transform to place them in the proper position of the virtual world. Then they must be subject to a view transform, that will select which part of the virtual world should be visible, and finally a projection

that will convert the selected area of the world to normalized screen coordinates, ready to be transformed into screen coordinates by the drivers.

World matrices should be composed combining translations, rotations and scaling. Rotations can be expressed either using Euler angles or quaternions. View matrices can be computed either with the look-at or with the look-in-direction techniques. Projections can be either parallel or perspective, but the user must be able to view the scene from different points.

Objects must be encoded as meshes, whose vertices are characterized by several parameters. These parameters should include the position, but also the direction of the normal vector to the surface, the texture coordinates, and maybe the tangent vector. Meshes should be either loaded from an external file, using formats such as OBJ or GLTF, or dynamically created with an algorithm. The corresponding vertex formats should be defined: in particular, an application can use several different formats at the same time, each one aiming at reducing the memory occupation, by offering only the attributes really needed for a specific mesh and rendering technique.

Navigation controls should be added, to allow the elements of the scene to be viewed either in first or third person. Navigation can occur using keyboard, mouse input, joystick or a mixture of different controls. The user should be able to interact with scene elements that will respond to their input – for example, moving some of the objects or some of the lights in the scene.

The final color of the pixels on screen should be determined using shaders. One or more pipelines should be defined to approximate the rendering equation in different ways. Each pipeline will support its own set of shaders, and it will start from the vertices specified in one of the introduced vertex formats. Shaders will communicate with the application using uniform buffers or push constants, where they will receive, for example:

- The world, view, and projection matrices (either separate or as a single entity).
- The position, direction, color and parameters for the lights.
- The textures required by the shaders.
- Other parameters necessary to compute the BRDF, such as the exponent in Phong or Blinn specular model, or a threshold in a Toon shader.
- The position of the viewer.
- And whatever is necessary to achieve controllable effects.

Several descriptors set layouts could be available, and different levels of bindings could be used to reduce the communication overhead between the CPU and the GPU. Vertex shaders will compute the position of the vertices in normalized screen coordinates, they will pass other parameters to the fragment shaders, such as texture coordinates and transformed normal vector directions, as well as other parameters, such as a light direction computed per vertex, or a color.

The final color of the pixels will be computed with fragment shaders, that will implement a model such as the Lambert or the Oren-Nayar diffuse, the Toon shader, or the Blinn, Phong or Ward specular models, or the Cook-Torrance, using either GGX or other distributions or geometric terms functions. They should use both direct lighting techniques, and indirect light approximations, such as ambient light or Image Base Lighting. Direct lighting can include directional sources, point sources, spotlights or other custom models.

The parameters of the material, such as diffuse and specular colors, emission, roughness, metalness, could be either specified in some uniform variables or read from the textures. Textures can also be used in advanced rendering techniques such as normal maps or detail textures.

The final quality of the images generated will also be considered: after all, it is a computer graphics course, and the final appearance is also very important. Although the most important thing will be the techniques being used, the overall size of the project will be considered, as well

as the introduction of extra elements, such as animation, basic physics simulation, intelligence in some algorithms, on-screen display, start-up menus, splash screens, scenes change, and so on.

Each project will require a set of 3D assets. An extensive set of assets will be provided, with details given on the WeBeep page of the course. They will include food, furniture, cars, buildings, vegetation (both modern and in medieval settings), simple daily life objects, dungeons, terrain, schools, hospital wards, and many others. Textures and PBR materials, as well a HDRI background will also be provided. However, the use of different assets either done or obtained by the students, is encouraged and will be positively evaluated during the presentation.